

PH331 : Computational Physics Lab

Chapter I

Atma Anand
SC12B156

Date: September 14, 2014

Department: Physical Sciences

1 Problem 1: Precision in your machine

Encode the following algorithm and run it to determine the smallest positive number that can be represented on the computer you are using:

```
input s <--- 1.0
for k=1,2,3,...,100 do
    s <--- 0.5 s
    t <--- s + 1.0
    if t <= 1.0 then
        s <--- 2.0 s
        output k-1, s
        stop
    endif
end
```

Do this for both single precision and double precision floating point numbers.

1.1 Program Code:

```
#include <stdio.h>

int main()
{
    double s=1.0,t;
    int i=1;
    for(i=0;i<100;i++)
    {
        s=s*0.5;
        t=s+1.0;
        if(t<=1.0)
        {
            s=s*2.0;
            printf("Double: %d \t %.30g \n",i,s);
        }
    }
}
```

```

        break;
    }
}
return 0;
}

```

1.2 Output:

```

[student@localhost SC12B156]$ ./prog1
Float:  23      1.1920928955078125e-007
Double: 52      2.2204460492503131e-016
[student@localhost SC12B156]$

```

Following are the observations:

- For the floating points the loop terminates at the 23rd iteration, i.e. *float* has 23 bits allocated for significand precision. This gives a best case precision of 9 significant digits but we see the value upto 17 decimal places.
- For the double-floating type the loop terminates at the 52nd iteration, i.e. **double** has 52 bits allocated for significand precision. This gives a best case precision of 17 significant digits and that is observed.

1.3 Discussion:

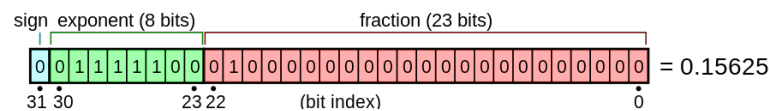


Figure 1: Binary Representation of Float Data (IEEE 754)

All data in a machine is stored as 1's and 0's. In the above exercise the smallest number of the form 2^{-n} which can be represented without complete truncation by the floating point data types, in their **decimal** form, is found by repeatedly dividing a variable by 2, starting with $1 (= 2^0)$. The point where the bits are forced to fully truncate is found by adding 1 to the variable and checking if its equal to 1. Once the variable has the required value, a simple addition of 1 to it will completely truncate the value of the variable to accommodate 1 and thus $1 + \text{variable} = 1$ is obtained.

When the variable has the value we seek the mantissa part has to have only zero values. The obtained result depends solely on the number of bits in the mantissa (say k) and is equal to 2^{-k} . **The exponent of 2^{-k} in the decimal form tells the no. of significant decimal digits of the variable.** In particular:

- For the floating point data type the 32 bits are split up as: 1 for sign, 23 for significand precision (as found out) and 8 bits (the rest) for the exponent part. Through the above exercise a precision of more than 8 digits is unexpected. However the variable can store any power of two within its range exactly till certain no. of digits (found to be 17 for C) and 2^{-23} has exactly 17 digits. Further unless forced any decimal value is assumed to be double which has a precision of 17 digits.
- For the double-floating type the 32 bits are split up as: 1 for sign, 52 for significand precision (as found out) and 11 bits (the rest) for the exponent part. This gives a precision of roughly 17 digits and that is observed.

2 Problem 2:

Evaluate the expression $y = \sqrt{x^2 + 1.0} - 1.0$ in two ways

(a) $y = \sqrt{x^2 + 1.0} - 1.0$

(b) $y = \frac{x^2}{\sqrt{x^2 + 1.0} + 1.0}$

for small values of x , $x = 0.1, 0.01, 0.001, \dots$. Determine the fractional error in both the methods of performing the subtraction.

Which method is superior, and why?

2.1 Program Code:

```
#include <stdio.h>
#include <math.h>

void main()
{
    double y,x=1.0,y1,d;
    int i=0;
    printf("\tx\t\t(a)\t\t(b)\t\tFractional Error\n\n");
    for (i=0;i<9;i++)
    {
        y=sqrt(x*x+1.0)-1;
        y1=(x*x)/(sqrt(x*x+1.0)+1);
        printf("%.1E\t%.10G\t%.10G\t%.5G\n",x,y,y1,(y1-y)/y1);
        x=x/10.0;
    }
}
```

2.2 Output:

```
[student@localhost SC12B156]$ ./prog2
```

x	(a)	(b)	Fractional Error
1.0	0.4142135624	0.4142135624	-1.3402E-016
1.0E-001	0.004987562112	0.004987562112	1.5478E-014
1.0E-002	4.999875006E-005	4.999875006E-005	1.9896E-012
1.0E-003	4.999998751E-007	4.99999875E-007	-1.1743E-010
1.0E-004	4.99999997E-009	4.999999987E-009	3.5775E-009
1.0E-005	5.000000414E-011	5E-011	-8.2765E-008
1.0E-006	5.000444503E-013	5E-013	-8.8901E-005
1.0E-007	4.884981308E-015	5E-015	0.023004
1.0E-008	0	5E-017	1

```
[student@localhost SC12B156]$
```

Following are the observations:

- For (comparatively) larger values of x , we see that the error between methods (a) and (b) is negligible.
- For extremely small values of x , (a) gives a truncated result of $y = 0$ while (b) gives $y = \frac{x^2}{2}$.

2.3 Discussion

Method (b) is much superior over (a). As x gets smaller and smaller, x^2 becomes negligible in front of 1 and ultimately gets truncated. i.e. $(x^2 + 1.0)^{0.5} \approx 1 + 0.5x^2 \approx 1$.

Hence at smaller values of x for (a): $y \rightarrow 0$ and for (b): $y \rightarrow 0.5x^2$. Thus (b) will give more accurate results.

3 Problem 3:

It is desired to calculate all integral powers of the number

$$x = \frac{\sqrt{5}-1}{2}$$

It turns out that the integral powers of x satisfy a simple recurrence relation

$$x^{n+1} = x^{n-1} - x^n$$

Show that the above recurrence relation is unstable by calculating x^{16}, x^{30}, x^{40} and x^{50} from the recurrence relation and comparing with the actual values.

3.1 Program Code:

```
#include <stdio.h>
#include <math.h>

void main()
{
    double y,x,x1,x2;
    int i;
    x1=(sqrt(5.0)-1.0)/2.0;
    y=x1;
    x2=x1*x1;
    printf("n\tRecursive\tActual \n\n");
    for(i=3;i<71;i++)
    {
        x=x1-x2;
        x1=x2;
        x2=x;
        if(i==16 || i==30 || i==40 || i==50)
        {
            x=pow(y,i);
            printf("%d\t%g\t%g\n",i,x2,x);
        }
    }
}
```

3.2 Output:

```
[student@localhost SC12B156]$ ./prog3
n          Recursive          Actual
16         0.000453104        0.000453104
30         5.37502e-007        5.3749e-007
40         5.83294e-009        4.37013e-009
```

```
50          1.79949e-007      3.55319e-011
[student@localhost SC12B156]$
```

Following are the observations:

- For the small values of n the Recursive relation appears to give results close to the actual value. Near the 40's the accuracy drastically reduces and after the 50's the recurrence relation becomes unstable.

3.3 Discussion

$x = \frac{\sqrt{5}-1}{2}$ is a root of the quadratic: $x^2 + x - 1 = 0$. Multiplying both sides by x^{n-1} and rearranging the recurrence relation is obtained. It is to be noted that x (a relative of the Golden Ratio) is an irrational and hence nor it or any of its natural number powers can be represented accurately in a digital system. Thus, every x^n for $n > 0$ has to be truncated while storing giving a error in the last significant (here 17th) digit. For the initial n the error introduced in the result is very small but as n increases more and more terms containing the truncated real numbers are required to be added to evaluated the result. A time comes for some n when the sum of the errors is comparable to the result and the solution becomes unstable.

4 Problem 4:

Given $x_n = (1/3)^n$, it turns out that the integral powers of $(1/3)$ satisfy a simple recurrence relation $x_{n+1} = (13/3)x_n - (4/3)x_{n-1}$ with $x_0 = 1$ and $x_1 = (1/3)$ Show that the above recurrence relation is unstable by calculating x_{15}, x_{30}, x_{40} and x_{50} from the recurrence relation and comparing with the actual values.

4.1 Program Code:

```
#include <stdio.h>
#include <math.h>
void main()
{
    double y,x,x1,x2;
    int i;
    x1=1.0/3.0;
    y=x1;
    x2=1.0/9.0;
    printf("n\tRecursive\tActual \n\n");
    for (i=3;i<51;i++)
    {
        x=(13.0/3.0)*x2-(4.0/3.0)*x1;
        x1=x2;
        x2=x;
        if(i==15 || i==30 || i==40 || i==50)
        {
            x=pow(y,i);
            printf("%d\t%G\t%G\n",i,x2,x);
        }
    }
}
```

4.2 Output:

```
[student@localhost SC12B156]$ ./prog4
n          Recursive          Actual
15         6.86307E-08         6.96917E-08
30        -1.13922            4.85694E-15
40        -1.19456E+06        8.22526E-20
50        -1.25259E+12        1.39296E-24
[student@localhost SC12B156]$
```

Following are the observations:

- For the small values of n the Recursive relation gives results close to the actual value. Near around $n=15$ the accuracy drastically reduces and around $n=30$ the recurrence relation becomes unstable.

4.3 Discussion

This problem is **similar** to the one above.

$x = \frac{1}{3}$ is a root of the quadratic: $x^2 - \frac{13}{3}x + \frac{4}{3} = 0$. Multiplying both sides by x^{n-1} and rearranging the recurrence relation is obtained. x is a recurring decimal no. and hence nor it or any of its natural number powers can be represented in powers of 2 without loss of information. Thus, every x_n for $n > 0$ has to be truncated while storing giving a error in the last significant (here 17th) digit. For the initial n the error introduced in the result is very small but as n increases more terms containing the truncated real numbers are required to be added to evaluated the result. A time comes for some n when the sum of the errors is comparable to the result and the solution becomes unstable. In this problem the solution becomes unstable much rapidly than in the previous problem. This occurs as numbers recurring themselves and > 1 are multiplied to the previous x_n terms and added to obtain x_{n+1} .

5 Problem 5:

The recurrence relation: $y_{n+1} = e - (n+1)y_n$

(where e is the base of natural logarithm) can be obtained from integration by parts to the integral

$$y_n = \int_0^1 x^n e^x dx$$

Show that the above recurrence relation is unstable by calculating y_{15} and y_{20} from the recurrence relation.

5.1 Program Code:

```
#include <stdio.h>
#include <math.h>

int n;

double func(double x)
{
    return (pow(x,n)*exp(x));
}
```

```

double integrate(int m, double a, double b)
{
    int i=0;
    double s=0.0, d=(b-a)/m;
    for (i=0; i<=m; i++)
        s+=func(a+d*i)*d;
    return s;
}

double rec(int k)
{
    if (k==1)
        return 1.0;
    if (k<=0)
        return exp(1)-1;
    return exp(1)-k*rec(k-1);
}

int main()
{
    printf("Enter n(-ve to exit): ");
    scanf("%d", &n);
    printf("\n\n\tIntegration\tRecursion\n");
    while(n>=0)
    {
        printf("%d\t%.10G\t%.10G\n", n, integrate(1e6, 0.0, 1.0), rec(n));
        scanf("%d", &n);
    }
    return 0;
}

```

5.2 Output:

```

[student@localhost SC12B156]$ ./prog5
Enter n(-ve to exit): 15 17 18 20 -1

```

n	Integration	Recursion
15	0.1604276681	0.1604958542
17	0.1434481334	0.1623630772
18	0.1362412501	-0.2042535616
20	0.1238051899	-129.2637081

Following are the observation(s):

- For values of $n \leq 15$ the Recursive relation gives results close to the actual value obtained by integration. Beyond that the accuracy drastically reduces and at merely $n=18$ recurrence relation becomes unstable.

5.3 Discussion

The recurrence relation is obtained readily by integrating the given integral by parts. It is to be noted that the relation contains $(n+1)y_n$. Now, e can only be represented precisely upto the 16th decimal place

by a double floating data type beyond which it is rounded and error is introduced. When the recurrence relation is used to find y_{n+1} , a term of $e.(n+1)!$ creeps in the sum. $18! \approx 6.4e15$, which multiplied to the error is comparable to the result. Beyond $n=18$ the recurrence relation becomes unstable beyond any resemblance. In this problem the relation becomes unstable much faster than the above problems due to the $n!$ term involved.

6 Problem 6:

Compute the dot product of the following two vectors

$x = [2.718281823, 3.141592654, 1.414213562, 0.5772156649, 0.3010299957]$

and

$y = [1486.2497, 878366.9879, 22.37492, 4773714.647, 0.000185049]$

Compute the summation in four ways

1. forward order Summation $\sum x_i y_i$ for $i = 1, \dots, n$
2. reverse order Summation $\sum x_i y_i$ for $i = n, \dots, 1$
3. largest to smallest order (add positive numbers in order from largest to smallest, then add negative numbers in order from smallest to largest and then add the two partial sums)
4. smallest to largest order (reverse order of adding in the previous method)

Use both single and double precision for a total of eight answers. Compare the results with the correct value $1.006571E-9$.

6.1 Program Code:

```
#include <stdio.h>
#include <math.h>

int sort(double *a, int n)
{
    int i, j, p;
    double temp;
    for (i=0; i<n-1; i++)
    {
        temp=a[i];
        p=i;
        for (j=i+1; j<n; j++)
            if (a[j]<temp)
            {
                temp=a[j];
                p=j;
            }
        temp=a[i];
        a[i]=a[p];
        a[p]=temp;
    }
    for (i=1; i<n; i++)
        if (a[i]>=0 && a[i-1]<0)
            return i;
}
```

```

void sortf(float *a, int n)
{
    int i,j,p;
    float temp;
    for (i=0;i<n-1;i++)
    {
        temp=a[i];
        p=i;
        for(j=i+1;j<n;j++)
            if(a[j]<temp)
            {
                temp=a[j];
                p=j;
            }
        temp=a[i];
        a[i]=a[p];
        a[p]=temp;
    }
}

int main()
{
    double s1,t[5],st1,st2;
    float sf1,tf[5],stf1,stf2;
    double x[]={2.718281823,-3.141592654, 1.414213562, 0.5772156649, 0.3010299957};
    double y[]={1486.2497,878366.9879,-22.37492,4773714.647,0.000185049};
    int i,n=5,k=0;
    s1=0.0; sf1=0.0;
    printf("Double\t\t\tFloat\n");
    for(i=0;i<n;i++)
    {
        t[i]=x[i]*y[i];
        s1+=t[i];
        tf[i]=(float)(x[i]*y[i]);
        sf1+=tf[i];
    }
    printf("Forward Sum:\n%.10E\t%.10G\n",s1,sf1);
    s1=0.0; sf1=0;
    for(i=n-1;i>=0;i--)
    {
        s1+=t[i];
        sf1+=tf[i];
    }
    printf("Reverse Sum:\n%.10G\t%.10G\n",s1,sf1);
    k=sort(t,5);
    sortf(tf,5);
    printf("\nAfter Sort:\n");
    st1=st2=0; stf1=stf2=0;
    for(i=0;i<k;i++)
    {
        st1+=t[i];
        stf1+=tf[i];
    }
}

```

```

    }
    for (i=k; i<n; i++)
    {
        st2+=t[i];
        stf2+=tf[i];
    }
    s1=st1+st2;
    sfl=stf1+stf2;
    printf("Forward Sum:\n%.10G\t%.10G\n", s1, sfl);
    st1=st2=0;
    stf1=stf2=0;
    for (i=k-1; i>=0; i--)
    {
        st1+=t[i];
        stf1+=tf[i];
    }
    for (i=n-1; i>=k; i--)
    {
        st2+=t[i];
        stf2+=tf[i];
    }
    s1=st1+st2;
    sfl=stf1+stf2;
    printf("Reverse Sum:\n%.10G\t%.10G\n", s1, sfl);
    return 0;
}

```

6.2 Output:

```

[student@localhost SC12B156]$ ./prog6
Double          Float
Forward Sum:
-7.4313859653E-006    -0.2499442995
Reverse Sum:
-7.431405265E-006    -0.2043457031

After Sort:
Forward Sum:
-7.431488484E-006    -0.25
Reverse Sum:
-7.431488484E-006    -0.25
[student@localhost SC12B156]$

```

Following are the observations:

- The forward sum and reverse sum give *different* results for the *same* dot product. After sorting the result is same however.
- For the double type the answer obtained has more significant digits (and hence accuracy) than the float data type.

6.3 Discussion:

The strange result of differing forward and backward sum arises due to:

- *The vast magnitude (exponent) difference of different terms,*
- *High amount of significant digits in the elements of the x and y array.*

The elements of x and y contain around 10 significant digits. Hence their multiplication will have around 20 significant digits and will have to be truncated. This no doubt leads to some error but that is negligible compared to the consequence of adding numbers of altogether different orders of magnitude, which results in drastic truncation of value of the number with smaller magnitude to match the exponent of the larger number. This is more pronounced when the alternative numbers are of opposing signs. Hence to reduce this error the array $x_i y_i$ is sorted and then added. In this case the numbers with comparable magnitudes are dealt with first and the negative and positive numbers are segregated in opposite sides. The order of summation was found to be of little consequence in a sorted $x_i y_i$ array. As expected due to the above reasons the answer $(-7.4314885E-6)$ is not even of the same order of the actual answer: $1.006571E-9$.